

Manual

PCC Module OPC Communication PCC-OPC 8.1

Version 1.0.23191

Last changed on: 09.09.2020

Copyright

©Copyright 2012 All rights reserved.

SAP® and R/3® are registered trademarks of SAP AG.

WINDOWS® is a registered trademark of Microsoft Corporation.

MPDV® and HYDRA® are registered trademarks of MPDV Mikrolab GmbH.

ORACLE® is a registered trademark of ORACLE Corporation, California, USA.

Copying and distribution of this documentation or any part thereof, for any purpose or in any form, is prohibited without prior written permission from MPDV Mikrolab GmbH.

The information contained in this documentation is subject to change without prior notice.

Contents

1	PCC-OPC Machine Connection - Overview	5
2	Protocol Module PCC-OPC.....	7
2.1	OPC communication	7
2.2	OPC server	9
2.3	System requirements	9
2.4	Connection test using the OPC explorer.....	9
3	Configurations	9
3.1	Conditions / quantities	9
3.2	Overview of file assignments.....	11
3.3	Configuration of the protocol module PCC-OPC.....	11
3.3.1	Section [service]	12
3.3.2	Section [instance x]	12
3.4	DNC with OPC (DNC setting data download/upload)	14
3.5	Sample configuration “opcmpdv.ini” file	18
3.6	Error codes and OPC client texts	19
4	Enhanced configurations (Beverage data collection).....	21
4.1	Counter configuration	21
4.2	Time synchronization	21
4.3	Nominal capacity / configured capacity.....	24
4.4	Status recording	25
4.5	Input of a reason code for the status	26
4.6	Send date and time of the OPC client	26
4.7	Bit recording for IDs of the type H / F / B	27
4.7.1	Automatic adding of entries up to BIT32	27
4.7.2	Simplified configuration for BIT recording	28
4.8	Alive signal.....	29
4.8.1	Alive signal by time request	29
4.8.2	Alive signal by status channel.....	31
4.9	Configuration of output IDs in the OPC client opcmpdv.ini.....	32
4.10	Alert status if connection between OPC server and UMPS is interrupted	32
4.11	Extended logging with counter channels	34

1 PCC-OPC Machine Connection - Overview

Summary

In the world of automation the OPC standard has spread more and more and is still advancing to establish a direct communication with machines. For further information on OPC (OLE for Process Control), please refer to the respective documentation or the website of the OPC foundation <http://www.opcfoundation.org>.

Fields of application

To connect machines with the HYDRA system, HYDRA provides an OPC client (PCC-OPC) that is integrated in the software. This client is integrated as protocol module into HYDRA-PCC (Process Communication Controller). Generally, all data fields (variables) of a machine, which are made available by an OPC server of the machine, can be read out using this client. This approach can be used for data from the following HYDRA modules

- MDE (counters/meters, cycles, statuses)
- PDV (process values)
- DNC (setting values)
- Scale values

As the PCC-ADP application adapter has been enhanced, other data (e.g. HYDRA-MPL or HYDRA-ADE data) as well as complex ways of communication may now also be processed.

Implementation notes

In general, communication is based on the exchange of data using defined variables in the OPC server. The variable contents of machines are provided as labeled data points/OPC items in the OPC server. The OPC client connects application programs with these variables. Thus, read and write accesses to OPC items are made available for application programs in a specific and transparent way. The OPC-DA specification in version 2.0 defines the data transport.

For further details on which data points can be provided by a specific control, please refer to the corresponding documentation about the control in question.

Integration

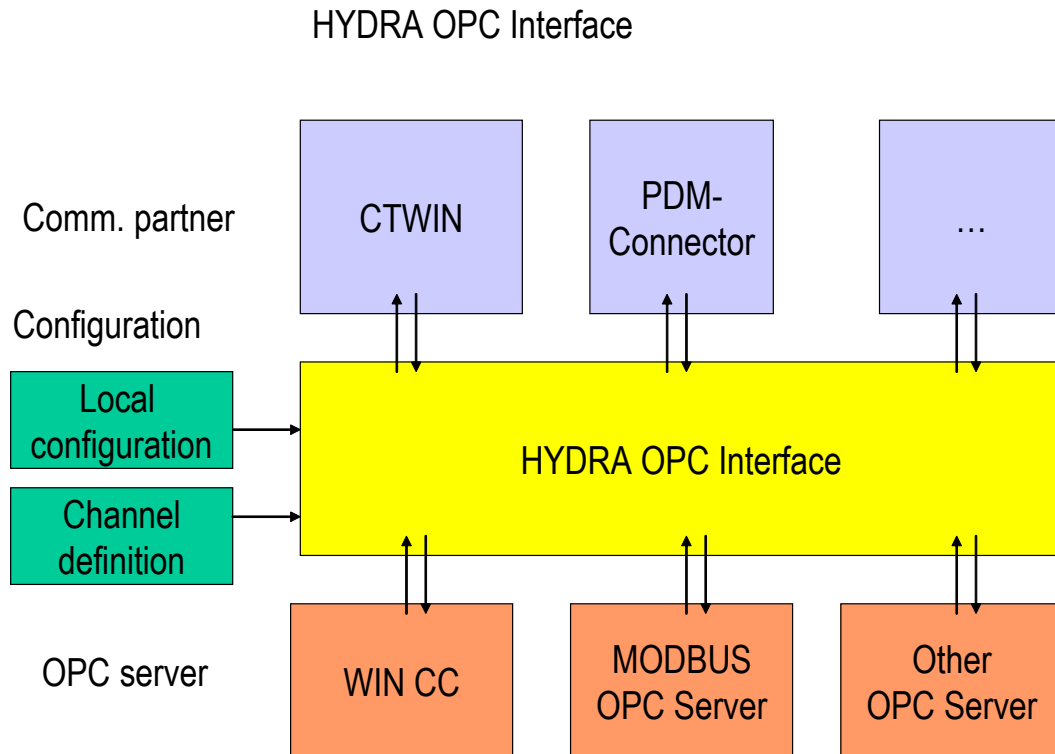
The protocol module is integrated in HYDRA PCC. Please also refer to the manuals dealing with the products [SCS-PCP](#) and [SCS-PCB](#)

Features

- Configurable OPC client for the communication with OPC servers in external systems, machines, equipment and controls to transfer OPC variables (e.g. machine statuses, process data, meter readings etc.) and NC data or setting parameters, required once for each connected machine, aggregate or control.

2 Protocol Module PCC-OPC

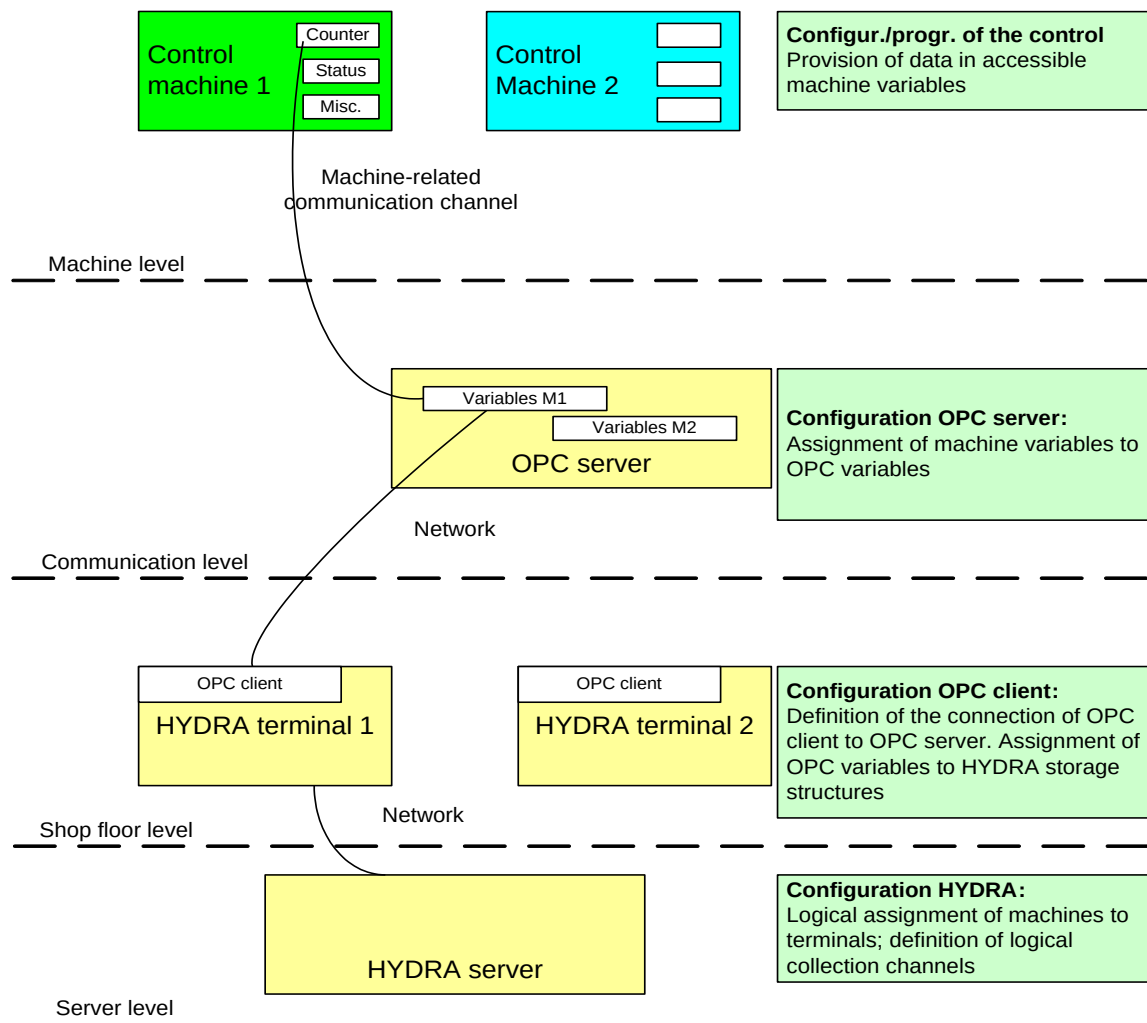
The protocol module PCC-OPC is an OPC-DA 2.0 client and represents the HYDRA OPC Interface.



2.1 OPC communication

OPC communication is based on the interaction of the following components:

Components of machine communication, configuration levels



Every level needs to be configured. Separation of responsibilities can be defined using such levels. In this context, MPDV is responsible for the shop floor and server level. The customer is responsible for the communication and machine level, which affects hardware as well as software. Consequently, the interface is exactly defined between OPC server and OPC client and, as a result, corresponds to the whole purpose of the OPC standardization idea.

On request, MPDV can offer or recommend appropriate OPC software for the communication level.

2.2 OPC server

The OPC server defines an open interface via which software components are able to exchange data. It reads in machine and process data of the machine and provides HYDRA applications with the data for processing. The OPC server may be installed on the same system as the client application (in this case it is PCC and CTWIN) but it is also possible to install it on a separate computer. By configuration it is defined which data are to be provided by the OPC server. The OPC server is not part of MPDV software but third-party software delivered by the manufacturer of the machine or a specialized system provider.

2.3 System requirements

There are no particular requirements that have to be met by the operating system as regards Win2000/XP. Distributed COM (DCOM) that is required is already part of the above-mentioned operating systems.

This description is based on the HYDRA release "MES-Weaver 2.0", which is required to establish the connection. Supported terminals are the CTWIN terminals as of the software version 7.2 on.

2.4 Connection test using the OPC explorer

An "OPC Explorer" (an OPC client) is very useful when the connection of the OPC server with the control is to be tested. It looks for available OPC servers within the network and represents the included value tables with variable contents. Thus, a correct connection can be ensured in advance even if there is no HYDRA configuration.

In general, any OPC Explorer can be used. In most cases, such clients are included by default when OPC servers are delivered.

3 Configurations

If HYDRA-PCC is in use a PCCDLL.DLL and its INI file of the same name need to be available in the local CTWIN directory. This file loads all other driver files for machine and scale connections. Here "opcmpdv.dll" is entered as driver for the HYDRA-OPC client.

3.1 Conditions / quantities

The following needs to be respected when using or configuring PCC-OPC:

	Number of characteristics / Process parameters for each	Number of machines	Fastest recording cycle
--	--	-----------------------	----------------------------

		machine		
PCC-OPC		Up to 300 or 400 pieces	Up to 20 pieces	4 seconds (1 second possible, but only for special applications)

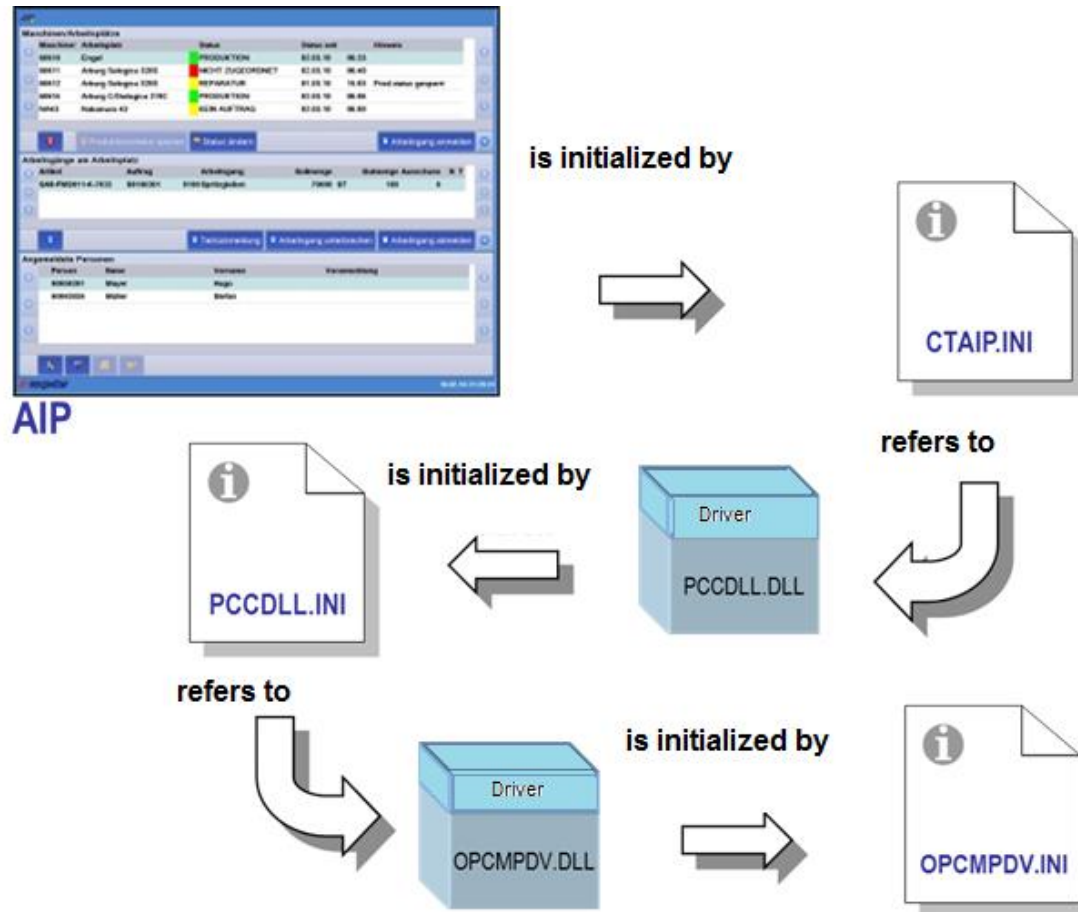
Please note:

There is not an “automatic” mode for data collection. Consequently, measured value data have to be collected or recorded in relation to time or pieces (i.e. cyclically).

In addition, offline collection does not transfer time stamps for measured values.

3.2 Overview of file assignments

The following diagram explains the connection between files for PCC configuration.



3.3 Configuration of the protocol module PCC-OPC

Configuration is defined in the "opcmpdv.ini" file. This document describes all possible parameters; however, not all of them are required for the relevant application and therefore need to be discussed with MPDV during the implementation process.

3.3.1 Section [service]

[SERVICE]

info=OPCMPDV.dll

interval=0

testmode=0

tracing=1 Activation of trace outputs.

TraceLevel=5 Trace level (5) is reasonable for outputs in a log file.

ExecuteQueue=1

ThreadBaseId=100

3.3.2 Section [instance x]

Communication instances can be built within the driver. Usually, one instance is defined for each OPC server or, a bit more detailed, for each machine connected. The section names can be chosen at will.

[OPC1] (Name of the first group)

Parameter	Function
HOSTNAME=192.168.10.116	IP address of the OPC server
SERVER={2E565242-B238-11D3-842D-0008C779D775}	If the host name is entered use the GUID identification of the OPC server process
SERVER=DSxPOpcSimulator.TSxOpcSimulator.1	The OPC name can be used with a local OPC
UPDATERATE=500	
OVERFLOW=30000	
OVERFLOW_MODE=0	
COUNTEREVENT=OFF	Counter events are not triggered automatically
DIGINEVENT=OFF	DIGIN events are not triggered automatically
VALEVENT=ON	Trigger event automatically if values are changed for V:XXXXX IDs (affects all V:XXXX IDs)
SETVALEVENTS=V:EGR:GUT V:EGR:AUS	Trigger events when changing a value for V:XXXX IDs (VALEVENT=ON must not be set in this case)

	An event is triggered for all V:XXXX IDs listed here when the input size is changed.
STATUSTIME=5	Timer in seconds. The server status is read in these time intervals. 10 seconds by default.
STATUSPROTOKOLL=OFF	Logging of the server status can be deactivated. The logging function is activated by default.
SAVE_COUNTER=ON	Saving of counter/meter readings when finishing and loading the saved counters/meters when being restarted. Counter/meter readings recorded in the control during the logoff phase of the terminal (OPC) are again collected when the terminal is restarted. Please also see the CTWIN documentation.
SETGROUP->C:C001=C:C002,C:C003,C:C008... SETGROUP->I:I001=I:I001,I:I010,I:I011,I:I012...	The IDs assigned in SETGROUP of a master ID may be distributed among other IDs. The assigned IDs are to be announced as channels but are not assigned an OPC item. C:C001=Item of the OPC for counter 1 C:C002=SETGROUP C:C003=SETGROUP
SETGROUPEVENT- >V:EGR:GUT=V:EGR:AUS,V:EGR:LEN...	Events are also triggered on other channels from the event of the master ID. (please note: not possible for counter channels)
SETTING->M:MSTAT@xxxx	Conditional setting of a status (special application)
SETTRIGGER_EVENT- >T:T001=C:C001,C:C002...	Several counters cause a trigger event.
UPDATERATE=500	The client (driver) synchronizes data with the OPC server within an interval of 500 msec.
OVERFLOW=30000	Machine counters overflow at 30000 and then restart with 1.

C:C00x=<OPC-Item> (Channel notation according to HYDRA convention)	OPC items for the collection of channels are entered as specified in the channel convention – please see the HYD-PCC documentation.
APPEND_DATE_TIME=ON	Date and time are attached to every data record of the OPC client. IDs and format are: BAS.DAT=MM/DD/YYYY BAS.ZEI=(seconds since midnight)

Please note for the server configuration:

A HOSTNAME= is not to be entered for a local OPC server, or better, the HOSTNAME is to be commented out by prefixing a semicolon ;HOSTNAME=. When the OPC server is connected via LAN, the ClassID No. of the OPC server is to be entered for Server=.

3.4 DNC with OPC (DNC setting data download/upload)

This function is available as of OPCMPDV.DLL version 7.2.1.44

This is a sample configuration:

DNC download file looks as follows Model structure of a DNC file
SOLLMENGE=20000 DRUCK=3.0 SOLLCYCL=4000
SOLLMENGE=20000 ; this is a comment line DRUCK=3.0 SOLLCYCL=4000
SOLLMENGE=20000 DRUCK=3.0 TEMP=80

SOLLCYCL=4000

Configuration of download items

Example for machine 4711 and 9999

D:DNC_{machine number}

{Machine number} The DNC machine number has to be entered here

The following configuration has been designed for the corresponding DNC machine and needs to be available.

(Required for the channel list within PCCDLL)

D:DNC_4711=NO_ITEM

D:DNC_9999=NO_ITEM

The IDs, as they are defined in the download file, have to be configured in the OPCMPDV.INI file.

<u>DNC file</u>		<u>OPCMPDV.INI</u>
SOLLMENGE=20000	→ OPCMPDV.INI	D:DNL_4711:SOLLMENGE
DRUCK=3.0	→ OPCMPDV.INI	D:DNL_4711:DRUCK
SOLLCYCL=4000	→ OPCMPDV.INI	D:DNL_4711:SOLLCYCL

The configuration of the OPCMPDV.INI file then looks as follows:

Download configuration: DNC IDs and OPC items

D:DNL_4711:SOLLMENGE=the corresponding OPC item

D:DNL_4711:SOLLCYCL= the corresponding OPC item

D:DNL_4711:DRUCK= the corresponding OPC item

Upload configuration : DNC IDs and OPC items

D:UPL_4711:SOLLMENGE= the corresponding OPC item

D:UPL_4711:SOLLCYCL= the corresponding OPC item

D:UPL_4711: DRUCK= the corresponding OPC item

Validity checking for DNC download

DNC_DNL_PLAUS=1

DNC_DNL_PLAUS	Function
DNC_DNL_PLAUS=1 Default value	A corresponding OPC item has to be available for all entries of the DNC file within the client configuration (opcmpdv.ini). Otherwise, it is cancelled with an error.
DNC_DNL_PLAUS=2	An entry needs to be available within the file for all entries of the client configuration. → At least which is configured
DNC_DNL_PLAUS=3	Everything that matches is transferred. The remainder not. (opcmpdv.ini)

Upload of setting data:

All identifiers available in the OPCMPDV.INI file → D:UPL_{machine number }

are read by the OPC during the upload and written in a text file.

Then the OPC client transfers the file into the respective directory. The terminal specifies the file name.

Download of setting data:

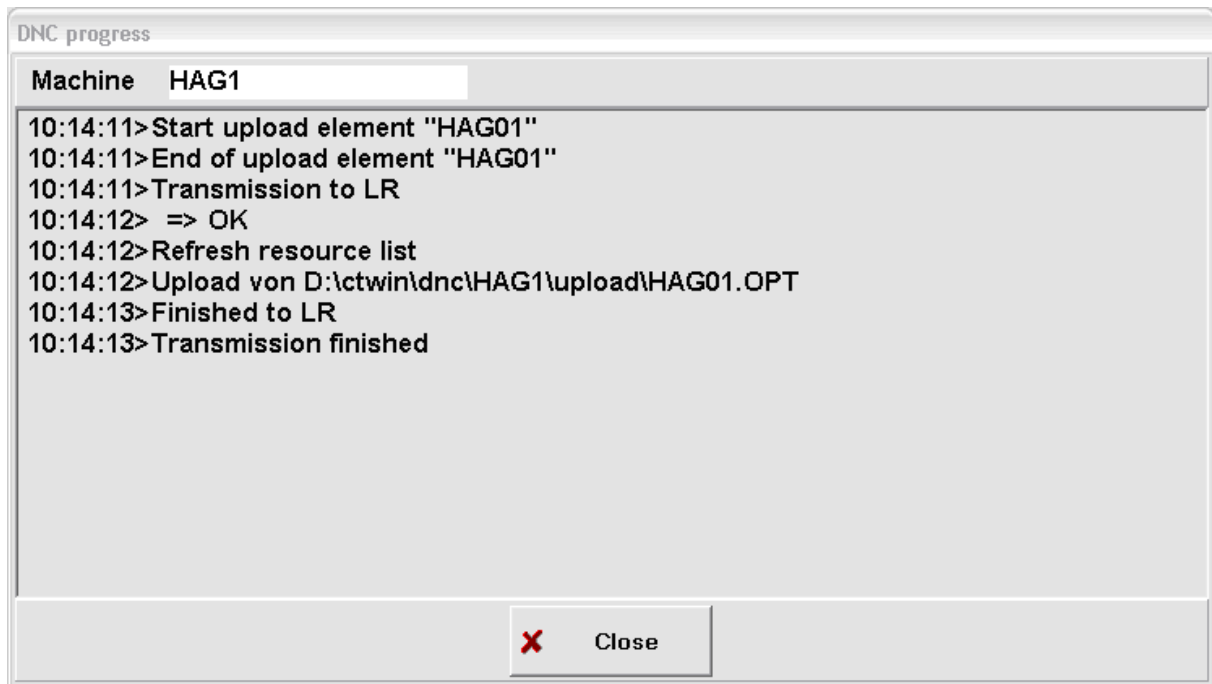
The OPC client reads the file provided by the terminal from the respective machine directory and sets all settings configured there to the available IDs configured in opcmpdv.ini. ,

Once it has been transferred correctly (without any error), the file is removed from the download directory.

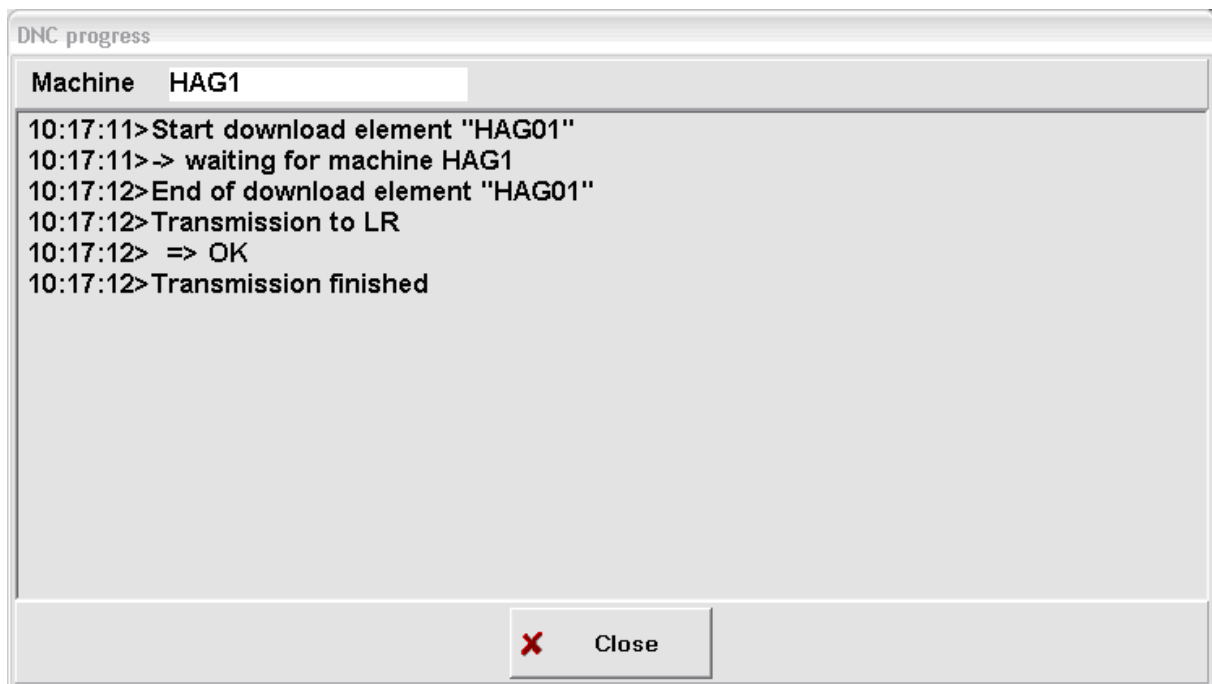
Please note:

If comments are included in a DNC file, a semicolon must precede the comment row. If this is not the case, this line leads to a misinterpretation at the OPC client. Comments are no longer included in the file after uploading.

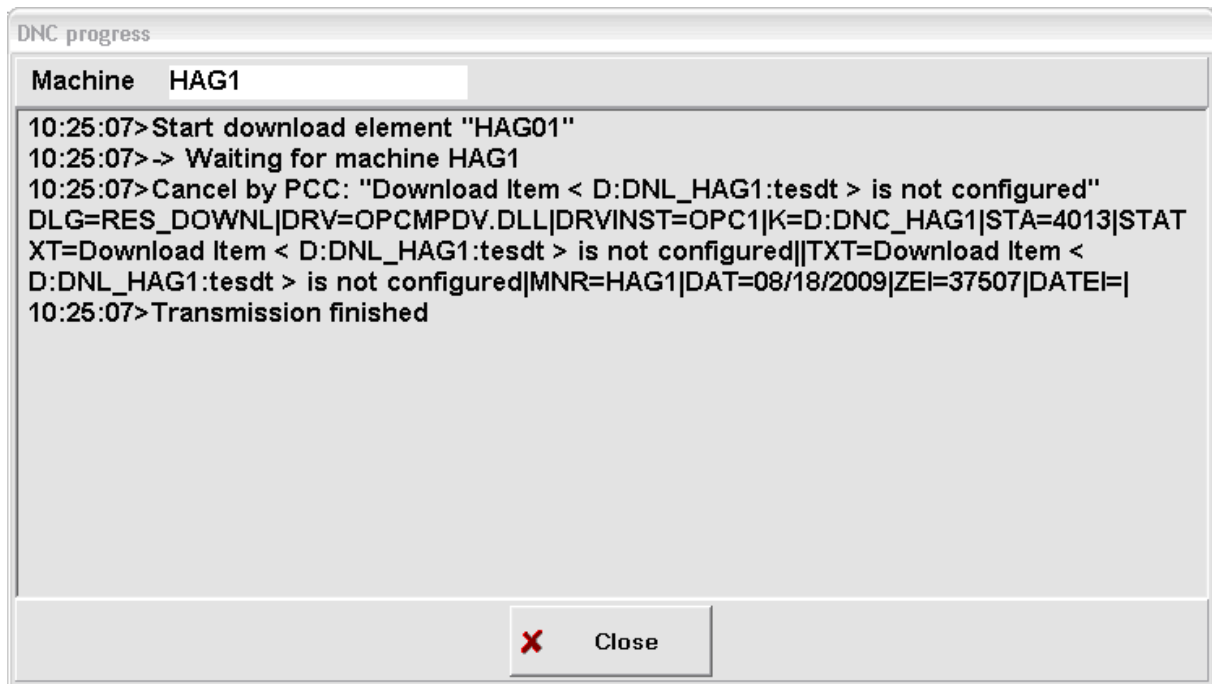
Error messages are displayed on the terminal.



Download is OK



Download is not OK



3.5 Sample configuration “opcmpdv.ini” file

PCC-OPC is configured in the “opcmpdv.ini” file.

Example:

```
[SERVICE]
info=OPCMPDV.dll
intervall=0
testmode=0
tracing=1
TraceLevel=5
ExecuteQueue=1
ThreadBaseId=100
Version=OPCMPDV-Version 7.2.1.11 / 28.06.2005
```

```
[OPC1]
;HOSTNAME=\\192.168.10.40
;SERVER=DSxPOpcSimulator.TSxOpcSimulator.1
SERVER={7904C302-AC19-11D4-9E1E-00105A4AB1C6}
UPDATERATE=500
OVERFLOW=30000
;OVERFLOW_MOD=0
```

← OPC instance number
← If OPC server on third-party system
← CLSID of OPC server

```
; RECONNECTTIME=10

; COUNTEREVENT=OFF
; DIGINEVENT=OFF
; SETVALEVENTS=V:EGR:GUT|V:EGR:AUS|
```

```
; I:I001=Input1
; M:MSTAT@maschine=MSTAT
; Z:Z001=Cycle time
C:C001=CT-UMPS/Counter_1
C:C002=CT-UMPS/Counter_2
C:C003=CT-UMPS/Counter_3
C:C004=CT-UMPS/Counter_4
I:I005=CT-UMPS/DI5
I:I006=CT-UMPS/DI6
O:O001=CT-UMPS/DO1
O:O002=CT-UMPS/DO2
O:O003=CT-UMPS/DO3
O:O004=CT-UMPS/DO4
O:O005=CT-UMPS/DO5
; V:V001=Value1
```

Signal configuration
(a ';' (semicolon) at the beginning of a line disables it)

```
POLL=0
POLL_I=2000
```

3.6 Error codes and OPC client texts

Code	Error text
4001	Channel not available
4003	Channel read error
4004	Channel bad quality
4005	DNC Upload configuration not available
4006	DNC Upload file name is empty
4007	DNC Error read OPC item

4008	DNC Directory does not exist
4009	DNC Error rename file
4010	DNC Error write file
4011	DNC Download configuration not available
4012	DNC Error write to OPC item
4013	DNC Download item is not configured
4014	DNC Download file does not exist
4015	DNC Download validity error <i>DNC_DNL_PLAUS</i>
4016	Wrong time format
4017	Error read time from item: %Item %Driver instance
4018	Error write time to item: %Item %Driver instance
4019	Error write countsync to item: %Item %Driver instance
4020	OPC server disconnect OPC server finished
4021	OPC server not active OPC server is not in the "running" status

The driver provides escalations as “E:{Hydra channel} error text“

Example: for status 100 E:S100 Channel bad quality

4 Enhanced configurations (Beverage data collection)

All configurations are performed in relation to the corresponding instance.

Additional configuration parameters for an OPC interfacing

4.1 Counter configuration

If the same counter numbers are used for the machines the machines also have to be indicated

C:C001@{machine number}={here: OPC item}

Example: Machine 4711 C:C001@4711=

4.2 Time synchronization

Please note: If it is possible to define a machine for a parameter the machine numbers are attached to the parameter by adding “@MachineNumber”.

Example for machine 4711

SENDCOUNTER@4711={OPC Item}

The parameter SENDLOCALTIME specifies the interval in which the OPC client sends time and sendcounter to the correspondingly configured OPC items.

But the parameter and its assigned OPC items have to be configured accordingly making sure data is output on the relevant OPC items.

SENDLOCALTIMER

Enables sending of the time and a counter to the OPC server.

The time interval is specified in minutes

SENDLOCALTIMER=0 time interval disabled

[Driver instance OPC1]

The time interval is specified in minutes

During this interval system time is synchronized with the OPC item.

SENDLOCALTIMER=1

SENDCOUNTER

[Driver instance OPC1]

Here: without machine assignment

SENDCOUNTER={here: OPC item}

Here: with machine assignment

SENDCOUNTER@{machine number}={here:OPC item}

Example: Machine 4711 SENDCOUNTER@4711=

A consecutive counter is output on an OPC item.

The parameter “SENDLOCALTIMER=x” determines the writing cycle for the OPC item.

The counter overflows at 65535 and restarts counting at 0.

The parameter SENDTIMEFORMAT specifies the time format that the OPC client outputs on the OPC item.

SENDTIMEFORMAT

Configures the date/time format for the communication.

[Driver instance OPC1]

SENDTIMEFORMAT=DATE-TIME

Possible date/time formats that can be set are:

DATE-TIME , UNSIGNED32 and TEXT

The format can be set once for each instance

The formats are structured as follows

Formats are written as well as read (READTIME)

The following formats are available

DATE-TIME	2010-06.24-14:45:00:000 YYYY-MM-DD-HH:MM:SS:MSec used S7 format
UNSIGNED32	Unix format since 1970
TEXT	20100624144500 YYYYMMDDHHMMSS

Greenwich Mean Time is supported

GMTTIME

[Driver instance OPC1]

GMTTIME=ON

If this parameter is set SENDTIME and READTIME are converted to GMT+0.

This affects the times communicating with the OPC server using SENDTIME and READTIME.

The SENDTIME parameter configures the OPC item on which the OPC client outputs the time.

SENDTIME

[Driver instance OPC1]

Here: without machine assignment

SENDTIME={here: OPC item}

Here: with machine assignment

SENDTIME@{machine number}={here: OPC item}

Example:

SENDTIMEFORMAT=DATE-TIME

SENDLOCALTIMER=1

SENDTIME@GLASABR={here: OPC item}

SENDTIME@4711={ here: OPC item}

SENDCOUNTER={ here: OPC item}

4.3 Nominal capacity / configured capacity

Parameters for configured capacity

M:EZY**[Driver instance OPC1]**

Here: without machine assignment

M:EZY={here: OPC item}

Here: with machine assignment

M:EZY@GLASABR={here: OPC item}

Parameter for nominal capacity

M:SZY**[Driver instance OPC1]**

Here: without machine assignment

M:SZY={here: OPC item}

Here: with machine assignment

M:SZY@GLASABR={here: OPC item}

4.4 Status recording

Status configuration always starts with the ID S:S

Status IDs ranging between S:S00001 and S:S65535 may be assigned.

Leading zeroes for HYDRA status IDs are removed in the MDE blade.

Status data records are sent without leading zeroes to the server.

[Driver instance OPC1]

Here: without machine connection

S:S100={here: OPC item for status 100}

S:S200={here: OPC item for status 200}

S:S300={here: OPC item for status 300}

Here: with machine assignment

S:S100@GLASABR={here: OPC item for status 100}

S:S200@GLASABR={here: OPC item for status 200}

S:S300@GLASABR={here: OPC item for status 300}

Start records are generated for every new status.

All statuses to be collected are configured this way.

Statuses requiring a status end posting:

Example: status 10000

The status 10000 switches from one status to the next. Consequently, the end of the status cannot be identified. Thus, the changeover to the next status finishes the previous status.

For this reason, the machine and the special status type is reset every time a changeover takes place.

Then a new status starts.

This affects statuses configured by S:S10000@{machine number}=10000;0.

At first a RES_STE with MOD=R is always sent

Dialog for reset:

Example: DLG=RES_STE|MOD=R| STATYP=10000|MNR=AUSP02|

Reset dialog string for a status type:

DLG=RES_STE|MOD=R|MNR=AUSP02|STATYP=10000|DLGMODE=AUTO|DAT=04/18/2011|ZEI=69619|USR=2120|OFF=J|

Status 7 start of dialog string for a status type:

DLG=RES_STB|MNR=AUSP02|STATYP=10000|STA=7|DLGMODE=AUTO|DAT=04/18/2011|ZEI=69619|USR=2120|OFF=J|

Generate end of the status (reset):

Example: Generate an end/completion record for status 2

If status 2 is available only a status reset is generated for this status type.

The status for which an end/completion record is generated is specified by „,{here: the status}\".

Example: ;2 for status 2

S:S10000@GLASABR={here: the OPC item for status 10000};2

4.5 Input of a reason code for the status

A reason configuration always starts with the ID S:S and must include an R.

Example: S:S100R=

Status IDs ranging between S:S00001 and S:S65535 may be assigned.

A reason code of status "R" can only be added to a status that has already been configured.

S:S100={here the OPC item for status 100}

S:S100R={here the OPC item for the reason assigned to status 100}

[Driver instance OPC1]

Here: without machine assignment

S:S100R={here OPC item for reason 100}

S:S200R={here OPC item for reason 200}

S:S300R={here OPC item for reason 300}

Here with machine assignment

S:S100R@GLASABR={here OPC item for reason 100}

S:S200R@GLASABR={here OPC item for reason 200}

S:S300R@GLASABR={here OPC item for reason 300}

4.6 Send date and time of the OPC client

Possible as of OPCMPDV.DLL

version 7.2.2.10

Date and time is added to each data record of the OPC client.

IDs and format are:

BAS.DAT=MM/DD/JJJJ

BAS.ZEI=(seconds since midnight)

Parameter that is to be used for the corresponding OPC instance:

APPEND_DATE_TIME=ON

4.7 Bit recording for IDs of the type H / F / B

As of version: opcmpdv.dll 7.2.2.4

Extract 32 Bit from a DWORD / HEX32.

BITs ranging between 1 and 32 are compared with each other and 0 or 1 is sent to the application when it comes to a change.

Used for IDs of the type H:H/F:F/B:B

BIT 1 .. 32 may be entered

The configuration has to be: **The ID ;@{2} behind the OPC item defines the BIT that is to be evaluated.**

BIT values ranging between 1 and 32 may be assigned.

Example : from Bit 1 to 32

H:H300@GLASABR=EINGANG.B90_0;@{1} here Bit 1

H:H303@GLASABR=EINGANG.B90_0;@{3} here Bit 3

H:H306@GLASABR=EINGANG.B90_0;@{6} here Bit 6

.. to Bit 32

H:H332@GLASABR=EINGANG.B90_0;@{32} here Bit 32

4.7.1 Automatic adding of entries up to BIT32

Simplified generation of BIT configurations

When restarting the driver, it generates the remaining BIT configurations for an ID.

All 32 BIT are always generated.

SETBITMODE->1= H:H300@GLASABR,H:H340@AUSP02

SETBITMODE->2 to n can be set for further entries, as the number of parameters that can be entered in a row is restricted.

SETBITMODE->2=H:H001@AUSP02,H:H040@AUSP02

SETBITMODE->3=.....

The 1 bit entry has already to be available in OPCMPDV.ini.

The other entries (until Bit 32) are added automatically in the INI file while the driver is restarted for the first time.

H:H300@GLASABR=EINGANG.B90_0;@{1} here Bit 1

is automatically created up to

H:H332@ GLASABR =EINGANG.B90_1;@{32} here Bit 32

H:H340@AUSP02=EINGANG.B90_1;@{1} here Bit 1

is automatically created up to

H:H372@AUSP02=EINGANG.B90_1;@{32} here Bit 32

4.7.2 Simplified configuration for BIT recording

Possible as of OPCMPDV.dll OPCMPDV.dll version 7.2.2.9

Please note: for the performance

This configuration should be used in any case for the beverage application to reduce the number of configurations included in the section.

From OPCMPDV.dll version 7.2.9 on F:F, H:H, B:B IDs may be configured and used for BIT recording.OPCMPDV.DLL automatically generates the required IDs when changing BITS and sends them to the application.

The automatically created IDs are always generated consecutively according to the configured HYDRA basic ID.

Please note: The simplified configuration can also be used for I:xxxx channels. The terminal only processes I:xxxx channels.

The following configuration can be used to reduce the number of item entries in the INI file.

Example:

The BIT number is specified by adding ;@32 ;@16 ;@8 to the OPC item.

Configuration:

Only the basic item and the first HYDRA ID are entered.

The other HYDRA IDs are consecutive as of the basic ID.

Please note: The next HYDRA ID that starts must not be within this range.

32 BIT recording HYDRA basic ID

F:F0500@WASCH02=here OPC item;@32

The channels

F:F0501@WASCH02=here OPC item;@32

to

F:F0531@WASCH02=here OPC item;@32

are generated and sent automatically by the OPC client when a BIT is changed

16 BIT recording HYDRA basic ID

F:F0531@WASCH02=here OPC item;@16

8 BIT recording HYDRA basic ID

F:F0531@WASCH02=here OPC item;@8

Other IDs that can be configured in this way:

B:B0800@WASCH02=test.Linie_R;@32

H:H0800@WASCH02=test.COM_L;@32

The BITs cannot be edited additionally by an additional configuration.

4.8 Alive signal

4.8.1 Alive signal by time request

The system time of the control may be requested cyclically to have an alive signal for checking. This method depends on the driver and is currently only available for OPC.

The system time is used as alive signal. The following configurations are required:

READLOCALTIMER=0 0 is the default value; timer is disabled. Otherwise, specification in minutes.

READTIME=<OPC-item> Read item for date and time

Please note: if it is possible to define a parameter for a machine the machine numbers are added to the parameter by “@Here: machine number”

READTIME@{machine number}={here: OPC item}

Example: Machine 4711 READTIME@4711=

The time format is specified as follows:

READTIMEFORMAT=UNSIGNED32 | TEXT | DATE-TIME

DATE-TIME is used as the time format for the S7 OPC interfacing.

READTIMEFORMAT=DATE-TIME

DATE-TIME: used for S7
Format: YYYY-MM-DD-HH:NN:SS:XXX
 Year;month;day;hour;minute;second:msec.

Example of S7: OPC explorer with S7 connection

Name	Quality	Time	Value
 S7/Zeit	good	09:35:11	2010-06-09-23:59:50.000
 S7/Name_STRING	good	09:35:11	Test

Information for the server as escalation message in the event of errors:

So far only available for OPC server.

If an error occurs when reading the OPC item, an error message is sent to the application.

- corresponds to the standard method of sending error messages (escalation)

E:READTIME='+ the OPC server text

E:READTIME=4016 wrong time format

E:READTIME=4017 no new time from machine

E:READTIME=4017 error read time from Item:'+ sltem

At the server error messages are entered in the dialog error log and if the escalation management module is also used they are also treated as escalation (event ERRPRO.ERROR_PROTOCOL_WRITTEN).

Note for machine head control: If a machine head control is used, the different machines can also be monitored in the machine head control and posted as communication status, see section 4.4 "Status recording" and section 4.8.2 "Alive signal by status channel".

4.8.2 Alive signal by status channel

If a machine is located, for example, behind a head control, the head control can monitor the machine and directly send the status to the OPC driver. This can be implemented as status channel. In addition to this, this status channel has to be monitored separately as it is no longer recorded if the head control breaks down. A “bad quality” status value is generated if the control fails. Moreover, it is assumed that all machine channels can no longer be used if the machine status “not reachable” is sent. Consequently, all postings/messages for the machine will be suppressed.

The driver provides the alive information as status channel: S:S123@<MNR>=ERR01

There is one such channel for each machine

Possible values: **Alive signal**

0 = Machine communication all right	ok
1 = Machine cannot be reached,	
2 = Channel malfunction at the machine,	
....	

The below-mentioned additional function is included as of opcmpdv.dll version 7.2.2.12:

The configuration → BADQUALITY-STATUS@S:S<channel number>@<machine>=<status value> enables channel monitoring, the generation of the bad quality status and the deactivation of the channel.

In this context S:S<channel number>@<machine> refers to the defined status channel and <status value> defines a status that is sent automatically if this channel fails, i.e. it sends “BAD QUALITY”.

Data collection is configured in the file: OPCMPDV.INI

Configuration is performed in the corresponding OPC instance.

Channel monitoring and definition of status values:

The alive channel is queried cyclically to identify any “BAD QUALITY” feedback that might be sent by the OPC server, provided that it does not trigger an event at the client.

The cyclic query is enabled by entering “READLOCALTIMER=x” .

The configuration is based on minutes, according to the configuration:

READLOCALTIMER=1 (also used for reading of the OPC server time)

Example for the machine: AUSP02

Processing is enabled by the configuration → BADQUALITY-STATUS@S:S123@AUSP02=xx.

xx stands for the status number that is sent when BAD QUALITY occurs at the OPC item.

Channel suppression:

Values must no longer be sent if the communication channels are not working properly, as these values are not realistic anymore. This is suppressed by the driver.

Values are only recorded if the status is 0 → machine communication is working properly (ok).

Sample configuration:

READLOCALTIMER=1

BADQUALITY-STATUS@S:S123@AUSP02=99

S:S123@AUSP02=enter the relevant OPC item for the alive signal.

If the channel sends "BAD QUALITY" the defined status is generated (99 in the example) and sent to the server. BADQUALITY-STATUS@S:S123@AUSP02=99

if the status value is <> 0 all events of the machine are blocked and not forwarded to the server.

4.9 Configuration of output IDs in the OPC client

opcmpdv.ini

Configuration to set outputs

Outputs are configured as follows

The ID always starts with **V:Vxxx**

xxx ← is always 3 characters long and identifies an output channel ranging between 001 and 999

Example for channel 1 machine 4711 and channel 10 for machine 4712

The channel configuration has to match the machine configuration in the database

V:V001@4711=OPC item for the output channel that is set

V:V010@4712=OPC item for the output channel that is set

Outputs are always set with signal 1 (ON)

Outputs are always reset with signal 0 (OFF)

4.10 Alert status if connection between OPC server and UMPS is interrupted

Provided that the OPC server has an item indicating that a connection to UMPS has been established, this item can be used as signal to change statuses on the terminal.

The function is available as of OPCMPDV.DLL version 7.2.2.41.

If the terminal knows the DIGIN channel configured in the INI file, the terminal switches to the status assigned to the DIGIN channel if the connection between UMPS and OPC fails.

Once the connection between OPC and UMPS has been re-established, the status switches to the currently applicable status sent by UMPS.

Please note:

Only applies if the connection between OPC server and UMPS is interrupted.

The channel is also set if the terminal is restarted and CT-UMPS cannot be reached.

The configuration is carried out for the corresponding instance in the file opcmpdv.ini

```
WATCHDOG=I:I500
```

```
INVERT_DIGIN=I:I500
```

```
I:I500=UMPS/Connected
```

The watchdog DIGIN and the item are entered. Only one DIGIN channel is entered.

All channels entered in WATCHDOG=I:I500,I:I600,I:I700 will be sent

I:I500=UMPS/Connected

Watchdog status DIGIN channels for several machines in one section (one CT-UMPS) are separated by comma.

WATCHDOG=<channel 1>, <channel 2>,.....

Example:

```
WATCHDOG=I:I500,I:I600,I:I700
```

Using this configuration, it is possible to invert DIGIN channels (for watchdog channels only)

```
INVERT_DIGIN=<channel 1>,<channel 2>,...
```

Example:

```
INVERT_DIGIN=I:I500,I:I002,I:I003,
```

4.11 Extended logging with counter channels

You can use this configuration to activate an extended logging of counter channels. You can globally activate all counters or only a specific counter.

The configuration is carried out for the relevant instance in the file opcmpdv.ini

```
COUNTER_LOG=ON
```

```
COUNTER_LOG->C:Cxx=ON
```

Notes:

- The current value (absolute) of the counter is saved.
- You can identify a driver restart via threadID in the time stamp.
- The log file has a size of approx. 50 MB. An "old" file is generated.
- The files are stored in ./spool (e.g. opcmpdv.opc1.c001.log).
- This function is supported from driver version 7.2.2.50.